

Consegna 6b - MPI

Il laboratorio si concentra sull'implementazione e l'analisi di tecniche di parallelizzazione su sistemi a memoria distribuita e condivisa (MPI e OpenMP). I test sono stati condotti sia in ambiente locale (PC personale) sia sul cluster di ateneo.

CPI

In questa fase abbiamo verificato l'efficienza dello scaling MPI scalando il problema su 1 nodo e 2 nodi.

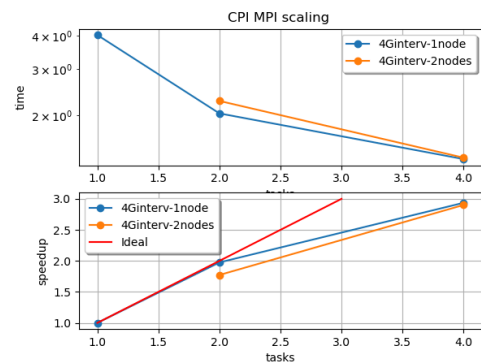
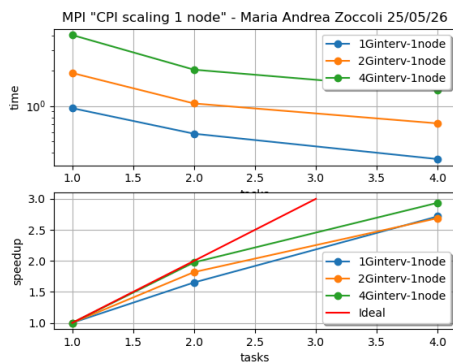
Test in Locale

```
andre@andrea:cpi$ python3 mpi_cpi-1n.py
```

	par	n	e2	t	nt	host
1	MPI	1.000000e+09	1.776357e-13	0.95786	1.0	andrea
4	MPI	1.000000e+09	1.079137e-13	0.58008	2.0	andrea
9	MPI	1.000000e+09	2.797762e-14	0.35255	4.0	andrea

```
andre@andrea:cpi$ python3 mpi_cpi-2n.py
```

	par	n	e2	t	nt	host
18	MPI	4.000000e+09	1.776357e-14	2.26943	2.0	wn1
23	MPI	4.000000e+09	5.506706e-14	1.38662	4.0	wn1



- Lo speedup a 4 task tra 1 nodo e 2 nodi è identico, questo è dovuto all'overhead dello stack TCP su interfaccia di loopback di WSL.
- All'aumentare dei task, il tempo di esecuzione si riduce proporzionalmente e la curva dello speedup segue quasi perfettamente la linea ideale.

Test Cluster HPC

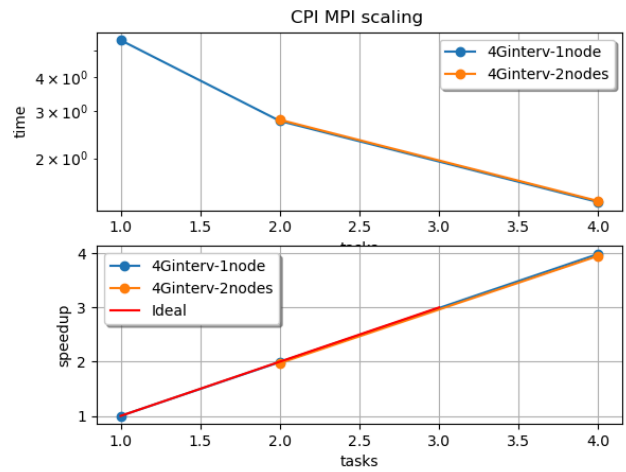
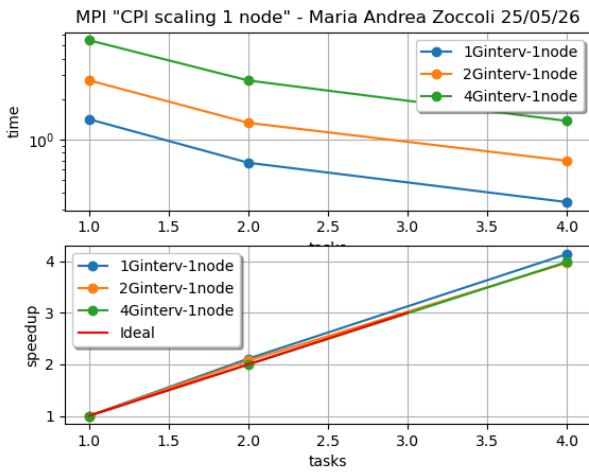
1 Nodo

MPI,	1000000000,	1.77635684e-13,	1.41868,	1,	wn20
MPI,	1000000000,	1.07913678e-13,	0.67256,	2,	wn20
MPI,	1000000000,	2.79776202e-14,	0.34279,	4,	wn20

2 Nodi

MPI, 1000000000, 1.07913678e-13, 0.69812, 2, wn20

MPI, 1000000000, 2.79776202e-14, 0.34899, 4, wn20



- Lo scaling su singolo nodo è ottima, passando a 2 nodi fisici notiamo una leggera perdita di efficienza iniziale dovuta al ritardo introdotto dall'invio dei messaggi attraverso la rete fisica, ma che viene compensata saturando i core.

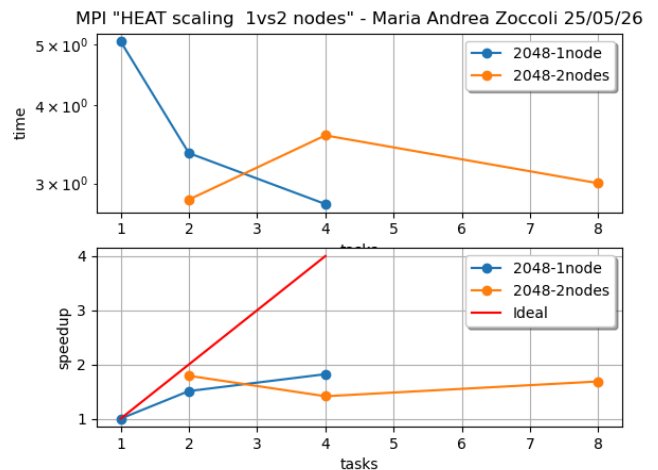
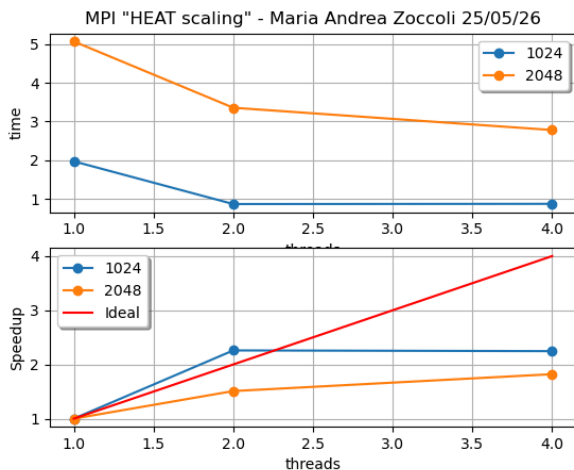
Heat

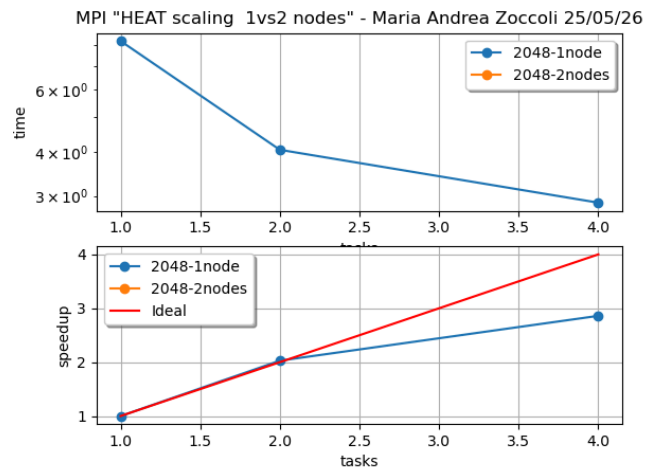
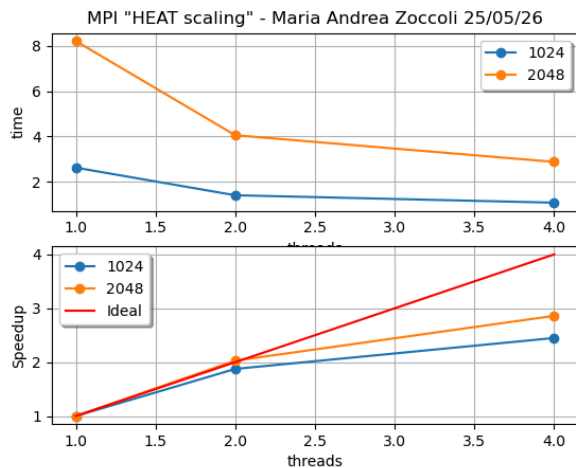
Scaling MPI Puro

Locale 1n

```
andre@andrea:heat$ python3 mpi_heat-1n.py
```

	par	nt	r	c	iter	t
3	MPI	1	2048	2048	1000	8.215990
4	MPI	2	2048	2048	1000	4.051449
5	MPI	4	2048	2048	1000	2.873043





Il problema è fortemente I/O Bound, poiché ad ogni iterazione i processi devono scambiarsi gli array delle ghost cells ai bordi del dominio.

Checkpointing (test locale)

```
andre@andrea:heat$ mpirun -n 4 ./mpi_heat_checkpoint -c 1024 -r 1024 -s 500
# File heatmap.txt doesn't exists
MPI, 4, 1024, 1024, 500, 0.432003

andre@andrea:heat$ mpirun -n 4 ./mpi_heat_checkpoint -c 1024 -r 1024 -s 500
# File heatmap.txt loaded
MPI, 4, 1024, 1024, 500, 0.428095
```

Per implementare il meccanismo di checkpointing il codice base `mpi_heat.c` è stato modificato in modo da poter salvare e ripristinare lo stato della computazione, garantendo la fault-tolerance.

- Caricamento dello stato iniziale:** all'inizio del programma, prima del ciclo iterativo, è stata introdotta la logica di caricamento: `load_heatmap(h_T_whole, WNX, WNY)`. Il rank master verifica l'esistenza di `heatmap.txt`, se presente ripristina i valori salvati, altrimenti procede con l'inizializzazione standard.
- Distribuzione dinamica:** una volta caricata la matrice globale dal file, il master utilizza `MPI_Scatter` per distribuire le rispettive porzioni ai vari rank, in modo che ogni processo possa ripartire con i dati aggiornati all'ultimo salvataggio.
- Salvataggio dello stato finale:** al termine delle iterazioni previste, dopo aver ricomposto la matrice aggiornata con `MPI_Gather`, il rank master esegue la funzione `save_heatmap(h_T_whole, WNX, WNY)` per serializzare su disco lo stato corrente, permettendo riavvii futuri tramite il flag `-s`.

Overlap Computation/Communication (test cluster)

```
[mariaandrea.zoccoli@ui01 heat]$ sbatch --partition=cpu_guest --qos=cpu_guest --nodes=1
--ntasks=4 --wrap="mpirun ./mpi_heat_overlap -c 2048 -r 2048"
Submitted batch job 2478922
# WNX:2048, WNY:2048, MAX_ITER:1000, MPI_RANK:0, MPI_SIZE:4, NX:2050, NY:514, wn21
MPI, 4, 2048, 2048, 1000, 1.413003
```

Per nascondere il costo delle comunicazioni di rete (I/O bound) è stato implementato l'approccio di Overlap Computation/Communication.

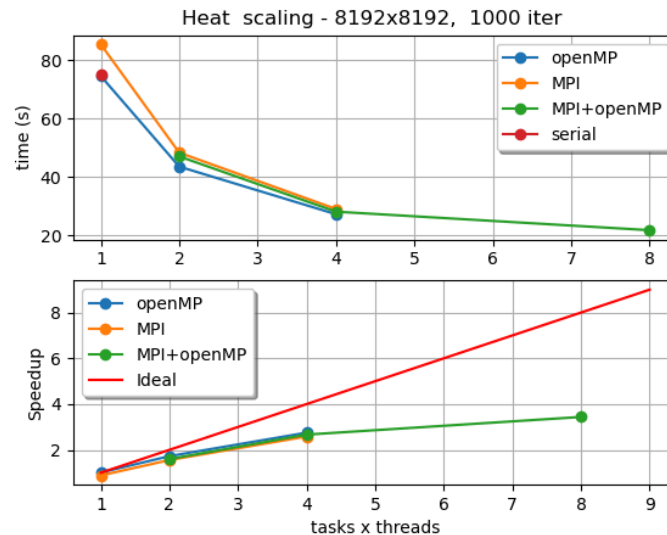
1. **Comunicazione non bloccante:** le chiamate sincrone `MPI_Sendrecv`, che bloccavano l'esecuzione in attesa dello scambio delle righe di bordo, sono state sostituite con primitive asincrone `MPI_Isend` e `MPI_Irecv`. Questo innesca il trasferimento dei dati in background.
2. **Sdoppiamento del dominio di calcolo:** la funzione nativa `Jacobi_Iterator_CPU` è stata sostituita dalle due varianti: `Jacobi_Iterator_CPU_internal` (aggiorna solo la il nucleo centrale della matrice) e `Jacobi_Iterator_CPU_external` (aggiorna solo la prima e l'ultima riga assegnate al task).
3. La struttura del ciclo iterativo è stata riorganizzata:
 - Vengono inviate le richieste asincrone (`Isend` / `Irecv`) per i bordi.
 - Mentre le schede di rete scambiano i dati, la CPU viene tenuta occupata lanciando `Jacobi_Iterator_CPU_internal`, poiché il nucleo della matrice non dipende dalle ghost cells esterne.
 - Al termine del calcolo interno, si attende esplicitamente la fine delle comunicazioni di rete tramite un costrutto di sincronizzazione (`MPI_Waitall`).
 - Ricevuti i bordi aggiornati dai vicini, si procede al calcolo delle righe vicine invocando `Jacobi_Iterator_CPU_external`.

Paradigma Ibrido (MPI+OpenMP)

Modifiche effettuate al file `mpi+omp_heat_scaling.slurm`:

1. **Allocazione slurm:** passato da `--cpus-per-task=1` a `--cpus-per-task=4` per fornire hardware reale ai thread OpenMP.
2. **Compatibilità compilatori:** sostituito il flag proprietario Intel `-ppn 1` con l'equivalente standard OpenMPI `-npernode 1`.
3. **Disattivazione core-pinning:** aggiunto il flag `--bind-to none` a `mpirun`. Di default, OpenMP forzava tutti i thread OpenMP su un singolo core fisico; queesto flag ha liberato i thread permettendo lo speedup.

	PAR	task	thr	r	c	iter	time
0	MPI+OMP	2	1	8192	8192	1000	47.060542
1	MPI+OMP	2	2	8192	8192	1000	28.186032
2	MPI+OMP	2	4	8192	8192	1000	21.863282



Dimostra che il paradigma Ibrido (MPI per inter-nodo, OpenMP per intra-nodo) abbatte i tempi a 21.8s, superando i paradigmi "puri" (blu e arancione, che si fermavano a ~27s) abbassando il sovraccarico di rete.

Fattorizzazione RSA

Modifiche effettuate al file `mpi_factorize.slurm` :

Sono state scommentate le direttive `#SBATCH --nodes=1` e `#SBATCH --ntasks-per-node=4` per forzare l'allocazione di tutti i task sullo stesso nodo fisico e minimizzare così la latenza di rete durante lo scambio del segnale di stop.

Locale

Senza early termination:

- Il task trova il risultato, ma gli altri procedono all'infinito.

```
task 1 processed block 250 in 17.7 sec. with result 1
task 1 processing block 246
...
task 2 processing block 169
```

Con early termination:

- Appena il task 1 trova il fattore, invia il messaggio. I task 0, 2 e 3 terminano il calcolo corrente e chiudono l'esecuzione

```
task 1 processed block 250 in 18.5 sec. with result 1
Task 1 found FAFE7315D
task 0 processed block 251 in 18.8 sec. with result 0
Task 0 found 0
task 2 processed block 245 in 14.6 sec. with result 0
Task 2 found 0
task 3 processed block 244 in 14.3 sec. with result 0
Task 3 found 0
```

Senza early termination:

- Nessuno si ferma e il programma continua ininterrottamente sprecando risorse preziose (infatti, il Task 2 trova il secondo fattore primo al blocco 213).

```
task 1  processed block 250 in 17.3 sec. with result 1
task 1  processing block 246
...
task 2  processed block 213 in 7.6 sec. with result 1
```

Con early termination:

- Il task 1 invia una `MPI_Send`, gli altri task la intercettano con `MPI_Test`, eseguono un break e si chiudono in pochi secondi.